1. High-Level Architecture Overview

The system runs on a GCP Ubuntu 22.04 Virtual Machine and consists of three core integrated layers:

1. Graph Database Layer (Neo4j): Stores entities and transactional relationships using the Bolt protocol (bolt://localhost:7687).
2. Distributed Data Processing Layer (PySpark): Performs scalable batch processing and aggregations on cleaned records, outputting JSON summaries to local storage.
3. Application & API Layer (Flask): Exposes a lightweight REST API on port 8000 that queries both the Neo4j graph and the PySpark output files dynamically.

2. Graph Database Schema (Neo4j)

The graph model represents drivers, their employers, drop-off destinations, and individual trips.

Node Definitions & Constraints

- Driver: Represents individual taxi drivers.
    - *Primary Property*: driver_id (Unique Constraint enforced)
- Company: Represents taxi companies / fleet operators.
    - *Primary Property*: name (Unique Constraint enforced)
- Area: Represents Chicago community drop-off areas.
    - *Primary Property*: area_id (Unique Constraint enforced)

Relationship Definitions

- (:Driver)-[:WORKS_FOR]->(:Company): Connects a driver to their affiliated company.
- (:Driver)-[t:TRIP]->(:Area): Connects a driver to a drop-off area containing trip metadata properties:
    - trip_id (String)
    - fare (Float)
    - trip_seconds (Integer)

3. Data Ingestion & PySpark Processing Pipeline

Ingestion & Cleaning (clean.py)

- Filters raw records down to essential columns (Trip ID, Taxi ID, Company, Pickup Community Area, Dropoff Community Area, Fare, Trip Seconds).
- Renames attributes to standard snake_case.
- Filters out invalid records where fare <= 0 or trip_seconds <= 0.
- Limits dataset to 10,000 clean records.

PySpark Aggregation (preprocess.py)

- Reads taxi_trips_clean.csv into a distributed Spark DataFrame.
- Feature Engineering: Computes fare_per_minute using: fare_per_minute=faretrip_seconds/60.0fare_per_minute=trip_seconds/60.0fare
- Aggregation: Groups data by company to calculate:
    - Total trip count (trip_count)
    - Average fare (avg_fare)
    - Average fare per minute (avg_fare_per_minute)
- Persistence: Coalesces output into a single partition JSON file within processed_data/.

4. REST API Specifications (Flask)

Endpoint	Method	Description	Response Format
/	GET	Health / status check	JSON status message
/api/drivers/<driver_id>/company	GET	Queries Neo4j to find the company associated with a driver ID	{"driver_id": "...", "company": "..."}

/api/companies/<company_name>/revenue	GET	Queries Neo4j to compute total fare revenue for a company	{"company": "...", "total_revenue": 0.00}
/api/spark/summary	GET	Reads and returns company-level metrics from PySpark output JSON	JSON Array of summary objects